A TFT-based Middle Product for Non-power-of-two Lengths

An implementation-oriented note based on TFT/ITFT recursion and transposition

1 Introduction

The *middle product* takes a length-n polynomial and a length-$(2n-1)$ polynomial and returns the n coefficients in the middle of their product. It is a standard subroutine in fast power-series algorithms, especially Newton iteration for inversion, division, and square roots.

For power-of-two lengths, the FFT solution is extremely clean: one uses three length-$2n$ transforms, just as in one ordinary FFT multiplication. Hanrot, Quercia, and Zimmermann also prove a more general statement: any bilinear multiplication algorithm yields a middle-product algorithm with the same bilinear complexity. That result tells us that a truncated-Fourier-transform solution *must* exist whenever polynomial multiplication can be done with the TFT.

The remaining question is practical rather than existential:

> How do we write down the non-power-of-two middle-product algorithm explicitly, in a way that looks natural and is close to a real implementation?

This note gives such a description. It is aimed at a reader who knows ordinary FFT-based multiplication well, but is less familiar with the TFT, the inverse TFT, and the transposition principle.

Two design choices guide the presentation:

1. We use *power-of-two* rather than *dyadic*, and *transpose* rather than *adjoint*.
2. We keep the pseudocode close to the recursive structure used in real code, in particular the split between the two inverse-TFT helper routines.

A final implementation remark up front: production libraries often stop the recursion early, use special base cases, and sometimes restrict to "admissible" sizes for efficiency. None of that is conceptually essential. For this write-up, every recursive routine simply goes all the way down to size 1.

2 Problem statement and notation

Let

$$y = (y_0, \dots, y_{n-1}) \in \mathbb{K}^n, \qquad z = (z_0, \dots, z_{2n-2}) \in \mathbb{K}^{2n-1},$$

and let

$$Y(x) = \sum_{i=0}^{n-1} y_i x^i, \qquad Z(x) = \sum_{k=0}^{2n-2} z_k x^k.$$

Write

$$Y(x)Z(x) = \sum_{r=0}^{3n-3} c_r x^r.$$

The length-n middle product is

$$\mathrm{MP}_n(y, z) = (c_{n-1}, c_n, \dots, c_{2n-2}) \in \mathbb{K}^n.$$

Throughout the note we use

$$\ell = 2n - 1, \qquad N = 2^p \geq \ell,$$

where N is the ambient power-of-two transform length used by the TFT.

We also use the reversal

$$\widetilde{y} = (y_{n-1}, \ldots, y_0),$$

and the obvious zero-padding convention: when a routine expects an array of length N, the active input occupies the first few positions and the remaining positions are initialized to zero unless stated otherwise.

Symbol	Meaning
n	target middle-product length
$\ell = 2n - 1$	full product length of two length-n inputs
$N = 2^p \geq \ell$	ambient power-of-two transform length
xn	active input length inside the forward TFT recursion
yn	active output length inside the forward or inverse recursion
w_j	butterfly twiddle at index j for the current size N

3 The power-of-two solution in terms of ordinary DFTs

Assume for the moment that $2n$ itself is a power of two, and let ω be a primitive $(2n)$-th root of unity. The classical DFT-based middle product is:

Procedure PowerOfTwoMiddleProduct(y, z)

1: $u \leftarrow \mathrm{DFT}_{2n,\omega}(\widetilde{y} \parallel 0^n)$
2: $v \leftarrow \mathrm{DFT}_{2n,\omega^{-1}}(z \parallel 0)$
3: **for** $i = 0$ to $2n - 1$ **do**
4: $\quad w_i \leftarrow u_i v_i$
5: **end for**
6: $r \leftarrow \dfrac{1}{2n} \mathrm{DFT}_{2n,\omega}(w)$
7: **return** the first n entries of r

The correctness argument is the standard wrap-around argument. If

$$C(x) = Y(x)Z(x) = \sum_{r=0}^{3n-3} c_r x^r,$$

then reducing modulo $x^{2n} - 1$ wraps the coefficients with index at least $2n$ back into the low-order part. The middle block

$$(c_{n-1}, \ldots, c_{2n-2})$$

stays untouched. After the reversal of y, those clean middle coefficients land in the first n positions after the final inverse transform.

The catch is that this argument really uses a transform of length $2n$. If $2n$ is not a power of two and we insist on radix-2 FFTs, we must round up to the next power of two. That can waste almost a factor of two.

4 Why the non-power-of-two case is subtler

A tempting idea is to embed the problem in a larger power-of-two FFT of length N and simply take the first $\ell = 2n - 1$ evaluation points. That does *not* reproduce the power-of-two construction:

- the power-of-two argument is tied to multiplication modulo $x^{2n} - 1$;
- a subset of the N-th roots of unity is not the same thing as the full set of $(2n)$-th roots;
- therefore "evaluate at the first ℓ FFT points and interpolate back" is not the right way to think about the middle product.

The correct viewpoint comes from transposition. If a multiplication algorithm can be written in bilinear form as

$$\mathrm{Mul}(a, b) = C^\top\big((Aa) \odot (Bb)\big),$$

then Hanrot, Quercia, and Zimmermann show that the corresponding middle product is

$$\mathrm{MP}(y, z) = A^\top\big((B\widetilde{y}) \odot (Cz)\big).$$

The pointwise multiplication stage stays the same; the surrounding linear maps reverse order and are transposed.

So the real task is this:

> Start from a concrete TFT-based multiplication algorithm, identify the linear maps around the pointwise products, and then write down their transposes explicitly.

5 The standard TFT and inverse TFT

5.1 What the TFT computes

Fix $N = 2^p \geq \ell$ and a primitive N-th root of unity ω. The TFT evaluates a polynomial at the first ℓ bit-reversed FFT points. Concretely, if

$$A(x) = a_0 + a_1 x + \cdots + a_{\ell-1} x^{\ell-1},$$

then

$$\mathrm{ForwardTFT}_{\ell,N}(a) = \big(A(\omega^{[0]_p}), A(\omega^{[1]_p}), \ldots, A(\omega^{[\ell-1]_p})\big).$$

Van der Hoeven proves that the forward TFT can be computed using at most

$$\ell p + N$$

additions/subtractions and at most

$$\left\lceil \frac{\ell p + N}{2} \right\rceil$$

multiplications by powers of ω. The cost therefore varies smoothly with ℓ, instead of jumping at powers of two.

5.2 Forward TFT recursion

The NTL-style routine is a general truncated transform with two length parameters:

- xn is the number of active input coefficients;
- yn is the number of transform values to compute;
- the ambient size is $N = 2^k$.

The routine works in place on an array $A[0..N-1]$. At entry, the first xn positions hold the coefficients and the rest are zero. At exit, the first yn positions hold the requested transform values.

Procedure ForwardTFTRec(A, yn, xn, k)

Input. $A[0..xn-1]$ contains coefficients; $A[xn..N-1] = 0$, where $N = 2^k$.

Output. $A[0..yn-1]$ contains the first yn TFT values.

$N \leftarrow 2^k$
if $N = 1$ **then**
  return
end if
$half \leftarrow N/2$
Let $X = A[0..half-1]$ and $Y = A[half..N-1]$
if $yn \le half$ **then**
  if $xn \le half$ **then**
    ForwardTFTRec$(X, yn, xn, k-1)$
  else
    $xn' \leftarrow xn - half$
    for $j = 0$ to $xn' - 1$ **do**
      $X[j] \leftarrow X[j] + Y[j]$ ▷ Only the left outputs are needed
    end for
    ForwardTFTRec$(X, yn, half, k-1)$
  end if
else
  $yn' \leftarrow yn - half$
  if $xn \le half$ **then**
    for $j = 0$ to $xn - 1$ **do**
      $Y[j] \leftarrow w_j X[j]$ ▷ Synthesize the right branch as (X, wX)
    end for
    ForwardTFTRec$(X, half, xn, k-1)$
    ForwardTFTRec$(Y, yn', xn, k-1)$
  else
    $xn' \leftarrow xn - half$
    for $j = 0$ to $xn' - 1$ **do**
      $u \leftarrow X[j]$, $v \leftarrow Y[j]$
      $X[j] \leftarrow u + v$
      $Y[j] \leftarrow w_j(u - v)$ ▷ Apply the butterfly $(X, Y) \mapsto (X + Y, w(X - Y))$
    end for
    for $j = xn'$ to $half - 1$ **do**
      $Y[j] \leftarrow w_j X[j]$ ▷ Right input is zero here
    end for
    ForwardTFTRec$(X, half, half, k-1)$
    ForwardTFTRec$(Y, yn', half, k-1)$
  end if
end if

For multiplication, we use the square case $xn = yn = \ell = 2n - 1$.

5.3 Inverse TFT: scaled recursion

The inverse side is slightly more subtle. The code is most naturally written in a *scaled* form: instead of returning the coefficient vector a, the recursive routines return Na, where $N = 2^k$ is the current transform size. This avoids dividing by 2 at every level of the recursion.

There are two mutually recursive helpers.

- ScaledInverseTFTEq handles the case $xn = yn$.
- ScaledInverseTFTFull handles the case $xn = N$.

The public inverse TFT is then obtained by one final multiplication by N^{-1}.

Procedure ScaledInverseTFTEq(A, yn, k)

Input. $A[0..yn-1]$ contains transform values. Entries beyond $yn-1$ are initially zero.

Output. $A[0..yn-1]$ contains $Na_0, \ldots, Na_{yn-1}$, where $N = 2^k$.

$N \leftarrow 2^k$
if $N = 1$ **then**
    return
end if
$half \leftarrow N/2$
Let $X = A[0..half-1]$ and $Y = A[half..N-1]$
if $yn \le half$ **then**
    for $j = 0$ to $yn-1$ **do**
        $X[j] \leftarrow 2X[j]$ ▷ Scaled inverse: postpone division by 2
    end for
    ScaledInverseTFTEq$(X, yn, k-1)$
else
    $yn' \leftarrow yn - half$
    ScaledInverseTFTEq$(X, half, k-1)$
    for $j = yn'$ to $half-1$ **do**
        $u \leftarrow X[j]$
        $X[j] \leftarrow 2u$
        $Y[j] \leftarrow w_j u$ ▷ Prepare the missing tail needed by the right recursion
    end for
    ScaledInverseTFTFull$(Y, yn', k-1)$
    for $j = 0$ to $yn'-1$ **do**
        $u \leftarrow X[j]$, $v \leftarrow Y[j]$
        $X[j] \leftarrow u + v/w_j$
        $Y[j] \leftarrow u - v/w_j$ ▷ Undo the last butterfly on the active prefix
    end for
end if

Procedure ScaledInverseTFTFull(A, yn, k)

Input. $A[0..yn-1]$ contains transform values; $A[yn..N-1]$ already contains the scaled coefficient tail needed by the recursion.

Output. $A[0..yn-1]$ contains $Na_0, \ldots, Na_{yn-1}$.

$N \leftarrow 2^k$
if $N = 1$ **then**
    return
end if
$half \leftarrow N/2$
Let $X = A[0..half-1]$ and $Y = A[half..N-1]$
if $yn \le half$ **then**
    for $j = 0$ to $yn-1$ **do**
        $X[j] \leftarrow 2X[j]$
    end for
    for $j = yn$ to $half-1$ **do**
        $X[j] \leftarrow X[j] + Y[j]$ ▷ Fold the known right-half tail into the left recursion
    end for
    ScaledInverseTFTFull$(X, yn, k-1)$
    for $j = 0$ to $yn-1$ **do**
        $X[j] \leftarrow X[j] - Y[j]$ ▷ Recover the correct active prefix
    end for
else
    $yn' \leftarrow yn - half$
    ScaledInverseTFTEq$(X, half, k-1)$
    for $j = yn'$ to $half-1$ **do**
        $u \leftarrow X[j] - Y[j]$
        $X[j] \leftarrow X[j] + u$
        $Y[j] \leftarrow w_j u$ ▷ This is $(X, Y) \mapsto (2X - Y,\ w(X - Y))$
    end for
    ScaledInverseTFTFull$(Y, yn', k-1)$
    for $j = 0$ to $yn'-1$ **do**
        $u \leftarrow X[j]$, $v \leftarrow Y[j]$
        $X[j] \leftarrow u + v/w_j$
        $Y[j] \leftarrow u - v/w_j$
    end for
end if

The public inverse transform is simply:

Procedure InverseTFT$(\hat{a}, \ell, p)$

$N \leftarrow 2^p$
Copy $\hat{a}$ into an array $A[0..N-1]$ and set the rest to zero
ScaledInverseTFTEq(A, ℓ, p)
for $j = 0$ to $\ell - 1$ **do**
    $A[j] \leftarrow N^{-1}A[j]$
end for
return $A[0..\ell-1]$

The scaled inverse has the same leading arithmetic cost as the forward TFT, and so does the public inverse, up to the final ℓ scalar multiplications by N^{-1}.

5.4 Polynomial multiplication with TFTs

Let $a, b \in \mathbb{K}^n$ and again set $\ell = 2n - 1$. Then TFT-based multiplication is:

Procedure TFTMULTIPLY(a, b, n)
1: $\ell \leftarrow 2n - 1$; choose $N = 2^p \geq \ell$
2: $u \leftarrow \text{ForwardTFT}(a, \ell, N)$
3: $v \leftarrow \text{ForwardTFT}(b, \ell, N)$
4: **for** $i = 0$ to $\ell - 1$ **do**
5: $\quad u[i] \leftarrow u[i] \cdot v[i]$
6: **end for**
7: **return** $\text{InverseTFT}(u, \ell, N)$

Compared to rounding all the way up to a full N-point FFT, the TFT saves work because its leading cost grows like $\ell p + N$, not Np.

6 The transposition principle in this setting

The multiplication algorithm above has the structure

$$\text{Mul}(a, b) = \text{InverseTFT}\big(\text{ForwardTFT}(a) \odot \text{ForwardTFT}(b)\big).$$

If we isolate the surrounding linear maps, this is exactly the situation covered by the transposition principle. The middle product is therefore obtained by:

1. reversing the order of the surrounding transforms;
2. transposing them;
3. keeping the pointwise multiplication stage unchanged;
4. reversing the short input y.

This gives the conceptual formula

$$\text{MP}_n(y, z) = \text{first}_n\Big(\text{TransposeForwardTFT}_{\ell,N}\big(\text{ForwardTFT}_{\ell,N}(\widetilde{y}) \odot \text{TransposeInverseTFT}_{\ell,N}(z)\big)\Big).$$

The rest of the note makes the two transposed primitives completely explicit.

7 The transpose of the forward TFT

7.1 Input/output specification

For fixed (yn, xn, N), let

$$F_{k,yn,xn} : \mathbb{K}^{xn} \to \mathbb{K}^{yn}$$

be the linear map computed by FORWARDTFTREC. Its transpose

$$F_{k,yn,xn}^{\top} : \mathbb{K}^{yn} \to \mathbb{K}^{xn}$$

is what we implement recursively.

Operationally, the routine works in place on an array of length N:

- at entry, $A[0..yn - 1]$ contains the input vector and the rest is zero;
- at exit, $A[0..xn - 1]$ contains the transpose output.

7.2 Pseudocode

The routine below is obtained by reading ForwardTFTRec backwards and transposing each local update.

Procedure TransposeForwardTFTRec(A, yn, xn, k)

Input. $A[0..yn-1]$ contains the transpose input; the rest is zero.

Output. $A[0..xn-1]$ contains the transpose output.

$N \leftarrow 2^k$
if $N = 1$ **then**
    return
end if
$half \leftarrow N/2$
Let $X = A[0..half-1]$ and $Y = A[half..N-1]$
if $yn \leq half$ **then**
    if $xn \leq half$ **then**
        TransposeForwardTFTRec$(X, yn, xn, k-1)$
    else
        $xn' \leftarrow xn - half$
        TransposeForwardTFTRec$(X, yn, half, k-1)$
        for $j = 0$ to $xn' - 1$ **do**
            $Y[j] \leftarrow Y[j] + X[j]$ ▷ Transpose of $X \leftarrow X + Y$
        end for
    end if
else
    $yn' \leftarrow yn - half$
    if $xn \leq half$ **then**
        TransposeForwardTFTRec$(X, half, xn, k-1)$
        TransposeForwardTFTRec$(Y, yn', xn, k-1)$
        for $j = 0$ to $xn - 1$ **do**
            $X[j] \leftarrow X[j] + w_j Y[j]$ ▷ Transpose of $X \mapsto (X, wX)$
        end for
    else
        $xn' \leftarrow xn - half$
        TransposeForwardTFTRec$(X, half, half, k-1)$
        TransposeForwardTFTRec$(Y, yn', half, k-1)$
        for $j = xn'$ to $half - 1$ **do**
            $X[j] \leftarrow X[j] + w_j Y[j]$ ▷ Transpose of the synthesized tail (X, wX)
        end for
        for $j = 0$ to $xn' - 1$ **do**
            $u \leftarrow X[j]$, $v \leftarrow Y[j]$
            $X[j] \leftarrow u + w_j v$
            $Y[j] \leftarrow u - w_j v$ ▷ Transpose of $(X, Y) \mapsto (X + Y, w(X - Y))$
        end for
    end if
end if

The public square case used later is just TransposeForwardTFTRec with $xn = yn = \ell$.

8 The transpose of the inverse TFT

8.1 Input/output specification

The inverse side is transposed at the same level of granularity as the code, so there are again two mutually recursive helpers.

- TransposeScaledInverseTFTEq is the transpose of ScaledInverseTFTEq. It maps a length-yn vector to a length-yn vector.
- TransposeScaledInverseTFTFull is the transpose of ScaledInverseTFTFull. It maps a length-yn vector to the full hybrid input required by the "full" inverse routine.

As before, the public transpose of the *actual* inverse TFT differs from the scaled one by one final multiplication by N^{-1}.

8.2 Pseudocode

Procedure TransposeScaledInverseTFTEq(A, yn, k)

Input. $A[0..yn-1]$ contains the transpose input; the rest is zero.

Output. $A[0..yn-1]$ contains the transpose output.

```
N ← 2^k
if N = 1 then
    return
end if
half ← N/2
Let X = A[0..half − 1] and Y = A[half..N − 1]
if yn ≤ half then
    TransposeScaledInverseTFTEq(X, yn, k − 1)
    for j = 0 to yn − 1 do
        X[j] ← 2X[j]                          ▷ Transpose of the scaled step X ← 2X
    end for
else
    yn′ ← yn − half
    for j = 0 to yn′ − 1 do
        u ← X[j], v ← Y[j]
        X[j] ← u + v
        Y[j] ← (u − v)/w_j                    ▷ Transpose of (X, Y) ↦ (X + Y/w, X − Y/w)
    end for
    TransposeScaledInverseTFTFull(Y, yn′, k − 1)
    for j = yn′ to half − 1 do
        X[j] ← 2X[j] + w_j Y[j]               ▷ Transpose of u ↦ (2u, wu)
    end for
    TransposeScaledInverseTFTEq(X, half, k − 1)
end if
```

Procedure TransposeScaledInverseTFTFull(A, yn, k)

Input. $A[0..yn-1]$ contains the transpose input; the rest is zero.

Output. The whole array $A[0..N-1]$ contains the transpose output.

$N \leftarrow 2^k$
if $N = 1$ **then**
    return
end if
$half \leftarrow N/2$
Let $X = A[0..half-1]$ and $Y = A[half..N-1]$
if $yn \leq half$ **then**
    for $j = 0$ to $yn - 1$ **do**
        $Y[j] \leftarrow Y[j] - X[j]$ ▷ Transpose of the final correction $X \leftarrow X - Y$
    end for
    TransposeScaledInverseTFTFull$(X, yn, k-1)$
    for $j = yn$ to $half - 1$ **do**
        $Y[j] \leftarrow Y[j] + X[j]$ ▷ Transpose of $X \leftarrow X + Y$ on the inactive tail
    end for
    for $j = 0$ to $yn - 1$ **do**
        $X[j] \leftarrow 2X[j]$
    end for
else
    $yn' \leftarrow yn - half$
    for $j = 0$ to $yn' - 1$ **do**
        $u \leftarrow X[j]$, $v \leftarrow Y[j]$
        $X[j] \leftarrow u + v$
        $Y[j] \leftarrow (u - v)/w_j$ ▷ Transpose of the active-prefix merge
    end for
    TransposeScaledInverseTFTFull$(Y, yn', k-1)$
    for $j = yn'$ to $half - 1$ **do**
        $t \leftarrow X[j] + w_j Y[j]$
        $Y[j] \leftarrow -t$
        $X[j] \leftarrow X[j] + t$ ▷ Transpose of $(X, Y) \mapsto (2X - Y,\ w(X - Y))$
    end for
    TransposeScaledInverseTFTEq$(X, half, k-1)$
end if

The public transpose of the actual inverse TFT is:

Procedure TransposeInverseTFT(u, ℓ, p)

$N \leftarrow 2^p$
Copy u into an array $A[0..N-1]$ and set the rest to zero
TransposeScaledInverseTFTEq(A, ℓ, p)
for $j = 0$ to $\ell - 1$ **do**
    $A[j] \leftarrow N^{-1} A[j]$
end for
return $A[0..\ell-1]$

9 The resulting middle-product algorithm

We can now state the non-power-of-two middle-product routine directly.

Procedure TFTMiddleProduct(y, z, n)

$\ell \leftarrow 2n-1$; choose $N = 2^p \geq \ell$
$u \leftarrow \text{ForwardTFT}(\widetilde{y}, \ell, N)$
$v \leftarrow \text{TransposeInverseTFT}(z, \ell, N)$
for $i = 0$ to $\ell - 1$ **do**
    $u[i] \leftarrow u[i] \cdot v[i]$
end for
$r \leftarrow \text{TransposeForwardTFT}(u, \ell, N)$
return the first n entries of r

This is the precise non-power-of-two analogue of the power-of-two FFT formula. The crucial point is that the long operand z does *not* go through an ordinary inverse TFT. It goes through the *transpose* of the inverse TFT. Likewise, the final interpolation step is replaced by the transpose of the forward TFT.

10 Running time and leading constants

Let

- $C_F(\ell, N)$ be the cost of the square-case forward TFT ($xn = yn = \ell$),
- $C_I(\ell, N)$ be the cost of the square-case scaled inverse routine ScaledInverseTFTEq,
- $C_{F^\top}(\ell, N)$ and $C_{I^\top}(\ell, N)$ be the costs of the two public transposed routines.

Van der Hoeven's bounds give the leading transform cost

$$C_F(\ell, N) \leq (\ell p + N) \text{ additions/subtractions}$$

and

$$\leq \left\lceil \frac{\ell p + N}{2} \right\rceil$$

twiddle multiplications, and the same leading bound holds for the inverse side.

The recursive transposed routines above have the same shape as the routines they transpose:

- the same recursion tree,
- the same number of active local updates,
- the same twiddle multiplications, just in reverse order.

So, to leading order,

$$C_{F^\top}(\ell, N) = C_F(\ell, N), \qquad C_{I^\top}(\ell, N) = C_I(\ell, N),$$

with only linear-time initialization, copying, scaling, and extraction outside the recursive kernels.

Therefore:

TFT multiplication of two length-n polynomials uses

$$2\, C_F(\ell, N) + C_I(\ell, N)$$

transform work, plus ℓ pointwise coefficient multiplications and one final length-ℓ scaling by N^{-1}.

TFT middle product of lengths $(n, 2n-1)$ uses

$$C_F(\ell, N) + C_{I^\top}(\ell, N) + C_{F^\top}(\ell, N) = 2\, C_F(\ell, N) + C_I(\ell, N)$$

transform work, plus the same ℓ pointwise coefficient multiplications and only linear-time extra bookkeeping.

In other words, the leading constant is the same:

> multiplication uses two forward transforms and one inverse transform; middle product uses one forward transform, one transpose of the inverse transform, and one transpose of the forward transform.

That is exactly what the transposition principle predicts.

11 Implementation notes

A few practical comments are worth recording.

1. **Recursing to size 1.** For exposition this is ideal. Real code may stop earlier and switch to a small base case.
2. **Admissible sizes.** The implementation in NTL includes additional logic around admissible sizes. That is a performance refinement, not a conceptual obstacle. The algorithms in this note remain correct without it.
3. **Twiddle tables.** All transposed routines use the same twiddle tables as the forward and inverse routines. Nothing new is needed beyond traversing them in the transposed order.
4. **The two inverse helpers are essential.** The inverse side is most naturally expressed with the pair ScaledInverseTFTEq/ScaledInverseTFTFull. Trying to hide that structure makes the inverse look much more mysterious than it really is.
5. **The transpose routines are not black magic.** They are obtained by reversing the recursive steps and transposing the local linear updates. In particular, the transpose of a pruning decision is usually just another pruning decision in the opposite direction.

12 Conclusion

The middle product for non-power-of-two lengths is not obtained by taking the first $\ell = 2n - 1$ points of a larger FFT and pretending that nothing changed. The correct algorithm comes from transposing a TFT-based multiplication algorithm.

Once the TFT multiplication is written in an implementation-friendly recursive form, the middle-product algorithm becomes quite natural:

- one ordinary forward TFT on the reversed short input,
- one transpose of the inverse TFT on the long input,
- pointwise multiplication,
- one transpose of the forward TFT,
- read off the first n coefficients.

The point of the present write-up is that all four transform primitives can be written in a uniform recursive style close to real code. That makes the transposition principle concrete enough to implement.

References

[1] G. Hanrot, M. Quercia, and P. Zimmermann, *The Middle Product Algorithm I*, Applicable Algebra in Engineering, Communication and Computing 14 (2004), 415–438.

[2] J. van der Hoeven, *The Truncated Fourier Transform and Applications*, ISSAC 2004.

[3] V. Shoup, *NTL FFT implementation* (`FFT.cpp`), with TFT/ITFT logic based in part on work of David Harvey.